

AGEX API Guide

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX API를 실제로 호출하는 Application Developer, Backend Developer, SDK Developer, Integration Engineer가 일관된 방식으로 사용할 수 있도록 하는 공식 API 사용 가이드이다.

본 문서는 앞서 정의된 **AGEX API Specification**의 계약을 구현 관점에서 설명하는 Companion Guide이다.

API Specification이 다음을 정의한다면:

- Resource Contract
- Endpoint Structure
- Header
- Authentication
- Error Envelope
- Pagination
- Idempotency
- Versioning
- Execution API
- Event/Webhook
- Security Boundary

본 API Guide는 다음을 설명한다.

- API를 어떻게 호출하는가.
- 인증 정보를 어떻게 전달하는가.
- Tenant Context를 어떻게 지정하는가.
- Resource를 어떻게 생성·조회·수정·Publish하는가.
- Agent와 Workflow를 어떻게 실행하는가.

- 비동기 Execution을 어떻게 추적하는가.
- Streaming은 어떻게 처리하는가.
- Error를 어떻게 해석하고 Retry하는가.
- Idempotency를 언제 사용하는가.
- ETag/If-Match를 어떻게 사용하는가.
- Event와 Webhook을 어떻게 연결하는가.
- SDK와 직접 API 호출을 어떻게 선택하는가.

본 문서는 특정 Programming Language에 종속되지 않는다.

2. API Guide의 공식 정의

AGEX API Guide는 AGEX의 Resource와 Runtime Capability를 외부 Application이 인증·Tenant·Version·Permission·Policy 계약을 준수하면서 안전하고 예측 가능하게 호출하기 위한 공식 API 사용 지침이다.

3. API 기본 원칙

AGEX API는 다음 원칙을 따른다.

- HTTPS Only
- API First
- Resource Oriented
- Versioned Contract
- Tenant Context Mandatory
- Authentication Required
- Authorization Server-Side
- Schema Validated
- Idempotency for Retry-Sensitive Operations
- Optimistic Concurrency

- Structured Error
 - Cursor Pagination
 - Async Execution for Long-Running Work
 - Streaming Separate from Execution State
 - No Direct Database Access
 - No Security Bypass Endpoint
 - No Raw Chain-of-Thought Exposure
-

4. API Base URL

기본 구조:

`https://{host}/api/{version}`

예:

`https://api.example.com/api/v1`

이후 예시는 `/api/v1`을 Base Path로 가정한다.

5. Content Type

기본 요청:

Content-Type: application/json

Accept: application/json

문자 인코딩은 UTF-8을 사용한다.

6. 인증

AGEX는 Deployment와 Client Type에 따라 다음 인증 방식을 지원할 수 있다.

- OAuth 2.x
- OpenID Connect

- API Credential
- Service Identity
- Workload Identity
- Signed Service Token
- mTLS

일반 Bearer Token 예:

Authorization: Bearer {access_token}

7. 인증과 권한의 차이

Authentication은 "누구인가"를 확인한다.

Authorization은 "무엇을 할 수 있는가"를 확인한다.

유효한 Token이 있다고 모든 Resource에 접근할 수 있는 것은 아니다.

Authenticated

≠

Authorized

8. Tenant Context

대부분의 Tenant Resource API에는 Tenant Context가 필요하다.

예:

X-AGEX-Tenant: ten_01H...

Token 또는 Session에서 Tenant가 유일하게 결정되는 경우 서버가 Context를 해석할 수 있지만, 여러 Tenant Membership을 가진 사용자의 경우 명시적 Tenant 지정이 필요할 수 있다.

9. Tenant Context 충돌

다음 정보가 서로 다르면 요청을 거부해야 한다.

- Authentication Token Tenant Claim
- Explicit Tenant Header
- Resource Tenant

서버가 임의로 하나를 선택해서는 안 된다.

10. 공통 Request Header

대표 Header:

Authorization: Bearer ...

Content-Type: application/json

Accept: application/json

X-AGEX-Tenant: ten_xxx

X-Request-ID: req_xxx

X-Correlation-ID: cor_xxx

Idempotency-Key: ...

If-Match: ...

모든 Header가 모든 Endpoint에서 필수는 아니다.

11. X-Request-ID

Client는 요청 추적을 위해 Request ID를 제공할 수 있다.

없으면 서버가 생성할 수 있다.

Response에서도 가능한 경우 반환한다.

문제 분석 시 중요한 식별자이다.

12. X-Correlation-ID

복수 API 요청과 Child Execution을 하나의 상위 업무 흐름으로 묶는 데 사용할 수 있다.

예:

Application Request

→ Agent Execution

→ Child Workflow

→ Plugin Call

모두 동일 Correlation Context를 가질 수 있다.

13. API Version

기본적으로 URL Version을 사용한다.

/api/v1

필요한 경우 Version 관련 Header를 추가로 제공할 수 있다.

API Version과 Resource Version은 서로 다르다.

14. API Version과 Resource Version

예:

API Version = v1

Agent Version = 7

/api/v1은 API Contract Version이며 Agent의 Published Version과 관계없다.

15. 기본 Response 구조

단일 Resource 예:

```
{  
  "id": "agt_123",  
  "type": "agent",  
  "tenant_id": "ten_123",
```

```
"name": "sample-agent",  
"version": 3,  
"revision": 1,  
"state": "active",  
"created_at": "2026-08-21T03:00:00Z",  
"updated_at": "2026-08-21T03:10:00Z"  
}
```

실제 필드는 Resource별 Schema를 따른다.

16. Resource ID

Resource ID는 일반적으로 서버가 생성한다.

Client는 ID의 내부 구조에 의존해서는 안 된다.

예:

agt_...

wfl_...

exe_...

plg_...

Prefix는 가독성을 위한 구현 방식일 수 있으며 계약에 명시된 범위에서만 의존한다.

17. Resource 생성

대표 Pattern:

POST /api/v1/agents

Body:

```
{  
  "name": "sample-agent",
```

```
"specification": {  
  "goal": "Handle a user request.",  
  "autonomy_level": "L1"  
}  
}
```

Response는 생성된 Draft Resource를 반환한다.

18. Resource 조회

GET /api/v1/agents/{agentId}

권한과 Tenant Scope가 적용된다.

존재하지만 접근 권한이 없는 Resource에 대해 보안상 404 또는 403 중 어떤 응답을 사용할지는 API Security Policy에 따른다.

19. Resource 목록 조회

GET /api/v1/agents

필터 예:

?state=active

?name=sample

지원되는 Filter는 Resource별 API Schema를 따른다.

20. Resource 수정

Draft Resource 수정 예:

PATCH /api/v1/agents/{agentId}

또는 API 계약에 따라 PUT을 사용할 수 있다.

Published Version 자체를 직접 수정하는 API를 제공하지 않는다.

21. Optimistic Concurrency

Resource 수정 충돌을 방지하기 위해 ETag 또는 Revision을 사용할 수 있다.

예:

Response:

ETag: "rev-17"

수정:

If-Match: "rev-17"

다른 Client가 먼저 수정했다면 Conflict를 반환한다.

22. Concurrency Conflict

예상 흐름:

Client A reads revision 17

Client B updates revision 18

Client A tries update with revision 17

→ Conflict

Client는 최신 Resource를 다시 조회하고 변경을 재적용해야 한다.

23. Publish

Agent Publish 예:

POST /api/v1/agents/{agentId}/publish

Publish 시 다음이 검증될 수 있다.

- Schema
- Dependency
- Permission

- Model Policy
 - Governance
 - Evaluation
-

24. Published Version

Publish 성공 시 새로운 Immutable Version이 생성된다.

예:

```
{  
  "id": "agt_123",  
  "version": 4,  
  "state": "published"  
}
```

25. Activate

Resource Lifecycle에 따라 Publish와 Activate가 분리될 수 있다.

예:

POST /api/v1/agents/{agentId}/activate

Published는 정의가 승인되었다는 의미이고 Active는 신규 실행에 사용할 수 있다는 의미일 수 있다.

26. Suspend

POST /api/v1/agents/{agentId}/suspend

Suspend 이후 신규 Execution을 제한할 수 있다.

기존 Running Execution 처리 방식은 Policy를 따른다.

27. Deprecate

POST /api/v1/agents/{agentId}/deprecate

Deprecated Resource는 신규 사용 제한 또는 Warning 대상이 될 수 있다.

28. Delete

DELETE /api/v1/agents/{agentId}

Dependency 또는 Lifecycle 상태에 따라 삭제가 거부될 수 있다.

Published Version의 Hard Delete는 일반 Draft보다 더 제한적일 수 있다.

29. Pagination

대규모 목록은 Cursor Pagination을 사용한다.

예:

GET /api/v1/agents?limit=50

Response:

```
{  
  "data": [],  
  "next_cursor": "cur_xxx"  
}
```

다음 요청:

GET /api/v1/agents?limit=50&cursor=cur_xxx

30. Offset Pagination 지양

대규모 또는 실시간 변화가 많은 Dataset에서는 offset=100000 같은 방식보다 Cursor를 우선한다.

이유:

- 성능
 - 일관성
 - 대규모 데이터 안정성
-

31. Sorting

지원되는 경우:

?sort=created_at

?sort=-created_at

정확한 표현은 API Specification에 따른다.

32. Field Selection

필요한 API에서는 응답 크기를 줄이기 위해 Field Selection을 지원할 수 있다.

예:

?fields=id,name,state

모든 Resource에서 의무 기능은 아니다.

33. Expansion

관련 Resource를 함께 반환해야 하는 경우 명시적 Expand를 지원할 수 있다.

예:

?expand=owner,model_profile

무제한 Nested Expansion은 제한해야 한다.

34. Filtering과 Authorization

중요:

GET all resources

→ filter tenant on client

방식을 사용하면 안 된다.

Authorization 및 Tenant Filtering은 서버 측에서 수행한다.

35. 표준 Error Envelope

대표 예:

```
{  
  "error": {  
    "code": "POLICY_DENIED",  
    "message": "The requested action is not permitted.",  
    "request_id": "req_123",  
    "details": {  
      "reason_code": "PLUGIN_ACTION_REQUIRES_APPROVAL"  
    }  
  }  
}
```

36. Error Code

Client는 가능하면 Human Message보다 Stable Error Code를 기준으로 처리한다.

예:

- AUTHENTICATION_REQUIRED
- PERMISSION_DENIED
- POLICY_DENIED
- VALIDATION_FAILED
- CONFLICT

- RESOURCE_NOT_FOUND
 - QUOTA_EXCEEDED
 - RATE_LIMITED
 - PROVIDER_UNAVAILABLE
 - EXECUTION_TIMED_OUT
 - INTERNAL_ERROR
-

37. HTTP Status와 Error Code

예시:

상황	HTTP
잘못된 입력	400
인증 실패	401
권한 없음	403
없음	404
충돌	409
Validation	422 가능
Rate Limit	429
서버 오류	5xx

정확한 Mapping은 API Specification을 따른다.

38. Validation Error

예:

```
{  
  "error": {
```

```
"code": "VALIDATION_FAILED",
"message": "Request validation failed.",
"details": {
  "fields": [
    {
      "path": "specification.autonomy_level",
      "code": "INVALID_ENUM"
    }
  ]
}
}
```

39. Error에서 민감정보 노출 금지

다음은 Error Response에 포함하지 않는다.

- Secret
- Access Token
- SQL Query 내부정보
- Internal Stack Trace
- Provider Credential

Internal Debug Detail과 Public Error를 구분한다.

40. Retry 정책

Client는 모든 Error를 자동 Retry하면 안 된다.

일반적으로 Retry 가능

- 429
- 일부 5xx
- Temporary Provider Error
- Network Error

일반적으로 **Retry** 금지

- 400
 - 401
 - 403
 - Validation
 - Policy Denied
-

41. Retry Backoff

권장:

Exponential Backoff

+

Jitter

서버가 Retry-After를 제공하면 우선 사용한다.

42. Idempotency

중복 실행 위험이 있는 POST 요청은 Idempotency-Key를 지원할 수 있다.

예:

Idempotency-Key: 1ea31f0e-...

43. Idempotency 사용 대상

특히 다음에 중요하다.

- Execution 생성
- Order
- External Side Effect
- 일부 Resource 생성

단순 GET 요청에는 필요하지 않다.

44. Idempotency 의미

같은 Tenant와 동일 Idempotency Context에서 같은 Key를 재사용하면 논리적으로 같은 Operation으로 처리할 수 있다.

Client는 서로 다른 업무 요청에 같은 Key를 재사용하면 안 된다.

45. Idempotency와 Request Payload 변경

같은 Key에 다른 Payload를 보내면 Conflict로 처리하는 것이 안전하다.

예:

key=abc

payload=A

then

key=abc

payload=B

→ Conflict

46. Agent API

대표 Endpoint 범주:

GET /agents

POST /agents

GET /agents/{agentId}

PATCH /agents/{agentId}

POST /agents/{agentId}/publish

POST /agents/{agentId}/activate

POST /agents/{agentId}/suspend

47. Agent 실행

편의 Endpoint 예:

POST /api/v1/agents/{agentId}/executions

Body:

```
{
  "input": {
    "message": "..."
  }
}
```

48. Agent Version 지정

Production에서는 가능하면 정확한 Version을 지정한다.

예:

```
{
  "agent_version": 4,
  "input": {
```

```
    "message": "..."  
  }  
}
```

Version을 생략했을 때 Active Version을 사용하는지 여부는 API Policy에 따른다.

49. Agent 실행 Response

비동기 실행:

```
{  
  "execution_id": "exe_123",  
  "state": "queued"  
}
```

Client는 Execution ID를 보관해야 한다.

50. Execution API

대표:

GET /executions/{executionId}

POST /executions/{executionId}/cancel

필요 시:

POST /executions/{executionId}/pause

POST /executions/{executionId}/resume

지원 여부는 Execution Type에 따라 다를 수 있다.

51. Execution 조회

GET /api/v1/executions/exe_123

예:

```
{
  "id": "exe_123",
  "state": "running",
  "target": {
    "type": "agent",
    "id": "agt_123",
    "version": 4
  },
  "created_at": "...",
  "started_at": "..."
}
```

52. Execution 완료

예):

```
{
  "id": "exe_123",
  "state": "completed",
  "output": {
    "result": "..."
  },
  "usage": {
    "model_input_tokens": 1200,
    "model_output_tokens": 300
  }
}
```

큰 결과는 Artifact Reference를 반환할 수 있다.

53. Long-Running Execution

장기 Task에서 다음 방식은 피한다.

POST

→ HTTP connection 20 minutes open

대신:

POST

→ execution_id

→ GET /executions/{id}

또는 Event/Streaming을 사용한다.

54. Polling

Polling 시 과도한 빈도를 피한다.

가능하면 Backoff를 사용하거나 Event/Webhook을 활용한다.

55. Streaming

Agent Output Streaming을 지원하는 Endpoint가 존재할 수 있다.

Streaming 방식은 구현에 따라:

- SSE
- WebSocket
- streaming HTTP

등을 사용할 수 있다.

공식 API 계약에 따라 선택한다.

56. Streaming과 Execution 상태

중요:

Streaming 연결 종료는 Execution 종료와 동일하지 않다.

Client Network가 끊어져도 서버 Execution은 계속 실행될 수 있다.

반드시 Execution API로 최종 상태를 확인할 수 있어야 한다.

57. Streaming Event 예

개념:

execution.started

output.delta

tool.requested

approval.required

execution.completed

Raw Chain-of-Thought Token Stream은 제공하지 않는다.

58. Workflow API

대표 Endpoint:

GET /workflows

POST /workflows

GET /workflows/{workflowId}

PATCH /workflows/{workflowId}

POST /workflows/{workflowId}/publish

POST /workflows/{workflowId}/activate

59. Workflow 실행

POST /api/v1/workflows/{workflowId}/executions

또는 Workflow Instance 전용 Contract가 있을 수 있다.

60. Workflow 입력

예:

```
{
  "workflow_version": 2,
  "input": {
    "subject": "...",
    "content": "..."
  }
}
```

Input Schema를 만족해야 한다.

61. Workflow 상태

Workflow Execution도 Execution Resource를 통해 확인한다.

필요 시 Step 상태가 포함될 수 있다.

```
{
  "state": "waiting_for_approval",
  "current_steps": [
    {
      "step_id": "approve-1",
      "state": "waiting"
    }
  ]
}
```

}

62. Approval API

대표:

GET /approvals

GET /approvals/{approvalId}

POST /approvals/{approvalId}/approve

POST /approvals/{approvalId}/reject

63. Approval 요청 조회

Approval에서 확인할 정보:

- Execution
 - Action
 - Target
 - Payload Digest
 - Risk
 - Requested At
 - Expires At
-

64. Approval 수행

예:

POST /api/v1/approvals/apr_123/approve

Body:

```
{  
  "comment": "Approved."  
}
```

```
}
```

승인 후 Runtime은 실제 Action Context를 다시 검증한다.

65. 완료된 Approval

완료된 Approval에는 approve 호출이 성공해서는 안 된다.

새 Approval Request가 필요하다.

66. Knowledge API

대표 범주:

/knowledge/collections

/knowledge/sources

/knowledge/documents

/knowledge/retrievals

구체 URI는 API Specification 기준으로 유지한다.

67. Knowledge Collection 생성

POST /api/v1/knowledge/collections

Body:

```
{  
  "name": "product-documents"  
}
```

68. Knowledge Source 등록

예:

POST /api/v1/knowledge/sources

Body에는 Source Type과 Configuration Reference가 포함될 수 있다.

Credential 원문을 Body에 장기 저장하는 방식은 사용하지 않는다.

69. Knowledge Ingestion

대용량 Ingestion은 비동기 Operation으로 처리할 수 있다.

예:

POST /api/v1/knowledge/sources/{sourceId}/ingestions

Response:

```
{  
  "operation_id": "op_123",  
  "state": "queued"  
}
```

70. Management Operation과 Execution

구분:

Execution

Agent/Workflow/Plugin/Model Runtime 작업.

Operation

Index Build, Export, Import 등 관리형 비동기 작업.

둘의 의미를 혼용하지 않는다.

71. Knowledge Retrieval API

직접 Retrieval이 필요한 경우:

POST /api/v1/knowledge/retrievals

예:

```
{  
  "collection_ids": ["knc_1"],  
  "query": "runtime recovery",  
  "limit": 10  
}
```

72. Retrieval Response

예:

```
{  
  "results": [  
    {  
      "content": "...",  
      "score": 0.91,  
      "citation": {  
        "document_id": "doc_123",  
        "source_id": "src_123"  
      }  
    }  
  ]  
}
```

권한 없는 문서는 결과 Candidate에도 포함되어서는 안 된다.

73. Memory API

대표:

/memories

/memory-proposals

/memory-retrievals

실제 Endpoint는 API Specification에 맞춘다.

74. Memory 직접 생성

일부 Memory Type은 API를 통한 생성이 허용될 수 있다.

예:

POST /api/v1/memories

반드시:

- scope
- owner
- retention
- classification

을 검증한다.

75. Memory Proposal

Agent가 제안한 Memory를 검토하는 흐름을 지원할 수 있다.

예:

POST /memory-proposals

GET /memory-proposals/{id}

76. Memory 삭제

DELETE /api/v1/memories/{memoryId}

삭제 성공 후 Recall에서 즉시 제외되어야 한다.

77. Plugin API

대표:

/plugins

/plugin-installations

/plugin-actions

78. Plugin 설치

예:

POST /api/v1/plugin-installations

Body:

```
{  
  "plugin_id": "plg_123",  
  "version": 2  
}
```

실제 설치 전 Permission Diff와 Compatibility Validation이 수행될 수 있다.

79. Installed와 Enabled

Plugin Installation이 존재한다고 Action 실행이 허용되는 것은 아니다.

Installed

≠

Enabled

≠

Authorized

80. Plugin Action 직접 호출

지원되는 경우:

POST /api/v1/plugin-actions/{actionId}/executions

하지만 일반 Application에서는 Agent/Workflow를 통해 실행하는 것을 더 권장할 수 있다.

81. Plugin Action Request

입력은 Manifest의 Input Schema를 따라야 한다.

예:

```
{  
  "input": {  
    "item_id": "123"  
  }  
}
```

Runtime이 Permission, Tenant, Credential 및 Side Effect를 검증한다.

82. Model API

대표 범주:

/models

/model-profiles

/model-executions

Application이 Provider API를 직접 사용할 필요를 줄이는 것이 목적이다.

83. Model Registry 조회

GET /api/v1/models

Filter 예:

?capability=tool_calling

?provider=...

84. Model Profile

Tenant의 Model Routing 설정을 Resource로 관리할 수 있다.

예:

```
{  
  "name": "production-default",  
  "required_capabilities": [  
    "structured_output",  
    "tool_calling"  
  ],  
  "allowed_provider_trust": [  
    "Private",  
    "Direct Approved"  
  ]  
}
```

85. Model 직접 실행

Agent 없이 직접 Model을 호출해야 하는 Use Case가 있으면 공식 Model Execution API를 사용할 수 있다.

예:

POST /api/v1/model-executions

이 경우에도 Model Gateway, Policy, Usage 및 Audit를 거친다.

86. Provider 직접 호출 금지

AGEX 내부 Application이 다음처럼 구현해서는 안 된다.

Application

→ Provider SDK

대신:

Application

→ AGEX Model API

→ Model Gateway

87. Marketplace API

대표 범주:

/marketplace/listings

/marketplace/packages

/marketplace/installations

/marketplace/subscriptions

Commercial API와 Runtime Permission API는 구분한다.

88. Marketplace 검색

예:

GET /api/v1/marketplace/listings?type=plugin

Response에는 다음이 포함될 수 있다.

- Publisher
- Version
- Trust
- Price

- Compatibility

89. Marketplace 구매

상업 시스템이 활성화된 경우:

POST /api/v1/marketplace/orders

구매 결과는 Entitlement를 생성할 수 있다.

90. Entitlement

Entitlement 조회 예:

GET /api/v1/entitlements

Entitlement는 설치 가능 권리이지 실행 권한이 아니다.

91. Marketplace 설치

구매 또는 허용된 Package:

POST /api/v1/marketplace/installations

이후 Tenant Installation과 Permission 절차가 수행된다.

92. Policy API

대표:

/policies

/policy-decisions

Policy 작성 권한은 높은 권한을 요구할 수 있다.

93. Policy 생성

POST /api/v1/policies

Policy Definition은 Machine-readable Schema를 따른다.

94. Policy Simulation

지원되는 경우:

POST /api/v1/policies/{policyId}/simulate

Production 상태를 변경하지 않고 예상 Permit/Deny 영향을 확인한다.

95. Policy Decision 조회

권한이 있는 운영자/관리자는 특정 요청의 결정 Record를 확인할 수 있다.

포함 가능:

- policy_id
- policy_version
- action
- result
- reason_code
- obligations

Raw Chain-of-Thought는 포함하지 않는다.

96. Governance API

대표 범주:

/governance/reviews

/governance/exceptions

/governance/evaluations

/governance/risk-profiles

97. Exception API

예:

POST /api/v1/governance/exceptions

필드:

- policy reference
- scope
- justification
- expiration
- compensating controls

영구 예외를 기본값으로 하지 않는다.

98. Audit API

GET /api/v1/audit/events

Filter 예:

?actor_id=...

?resource_id=...

?action=...

?from=...

?to=...

Audit 수정 Endpoint는 일반적으로 제공하지 않는다.

99. Usage API

GET /api/v1/usage

Dimension 예:

- tenant

- resource
 - model
 - plugin
 - execution
-

100. Usage와 Billing

Usage API는 운영 사용량을 제공한다.

Invoice나 Commercial Ledger는 별도 Billing Domain을 사용할 수 있다.

Usage Event 자체를 Client가 임의 수정해서는 안 된다.

101. Artifact API

대표:

/artifacts

/artifacts/{artifactId}

대규모 결과나 파일을 처리하는 데 사용한다.

102. Artifact Upload

실제 구현에 따라:

- Multipart Upload
- Pre-signed Upload
- Connector Upload

를 사용할 수 있다.

업로드 완료 후 서버 Validation이 필요하다.

103. Artifact Download

다운로드 권한은 Resource/Tenant Permission을 다시 확인한다.

영구 Public URL을 생성하지 않는다.

104. Event API

대표:

/event-subscriptions

/events

Application은 관심 Event를 구독할 수 있다.

105. Event Subscription

예:

```
{
  "event_types": [
    "execution.completed",
    "execution.failed"
  ],
  "target": {
    "type": "webhook",
    "webhook_id": "wh_123"
  }
}
```

106. Event Delivery

기본적으로 중복 전달 가능성을 고려한다.

Consumer는 Event ID를 기준으로 Idempotent 처리해야 한다.

107. Webhook 등록

POST /api/v1/webhooks

설정:

- endpoint
- subscribed events
- secret reference
- enabled

108. Webhook Signature

Webhook Receiver는 Signature를 검증해야 한다.

검증 대상:

- Payload
- Timestamp
- Signature

Replay Protection을 적용한다.

109. Webhook Retry

Receiver가 Temporary Failure를 반환하면 재전송될 수 있다.

무한 재전송하지 않는다.

최종 실패 시 DLQ 또는 Delivery Failure 상태를 제공할 수 있다.

110. Webhook SSRF 방어

Webhook URL 등록은 서버 측 검증 대상이다.

Private/Internal Network로의 무제한 Callback을 허용하지 않는다.

111. Rate Limit

API는 Client/Tenant/Endpoint별 Rate Limit을 적용할 수 있다.

429 응답 시 Retry-After가 제공될 수 있다.

112. Quota와 Rate Limit 차이

Rate Limit

짧은 시간의 호출 속도 제한.

Quota

기간 또는 전체 사용량 제한.

둘은 별개의 개념이다.

113. Request Size Limit

대규모 Payload는 Artifact를 사용한다.

API에 수백 MB JSON을 직접 전송하지 않는 것이 원칙이다.

114. Timeout

Client HTTP Timeout과 Server Execution Timeout을 구분한다.

예:

HTTP Client Timeout = 30 sec

Execution Timeout = 30 min

Client 연결이 끊어져도 Execution이 계속될 수 있다.

115. Client Timeout 전략

장기 Execution 생성 요청은 빠르게 Execution ID를 반환해야 한다.

Client는 실행 전체 Duration을 HTTP Timeout으로 잡지 않는다.

116. Authentication Token 만료

Long-running Execution 도중 사용자 Access Token이 만료되더라도 이미 승인된 Execution Identity가 Runtime에서 관리될 수 있다.

그러나 신규 관리 Action에는 새로운 인증이 필요하다.

117. Service-to-Service API

Backend Integration에서는 Workload 또는 Service Identity를 사용한다.

사용자 Token을 장기 Backend Credential처럼 재사용하지 않는다.

118. Least Privilege Client

Service Credential에는 필요한 API Scope만 부여한다.

예:

agent:execute

execution:read

만 필요한 서비스에 tenant:admin을 부여하지 않는다.

119. Scope

Authentication System이 Scope를 지원하는 경우 예:

agents.read

agents.execute

workflows.read

executions.read

실제 Scope 이름은 Identity Contract를 따른다.

120. API Credential Rotation

장기 API Credential은:

- Secret Store
- Expiration
- Rotation
- Revocation

정책을 적용한다.

121. Secret API

Secret Value 관리 API는 고위험 Endpoint이다.

가능하면 Secret Value를 Response에서 재노출하지 않는다.

예:

POST secret

→ reference returned

이후:

GET secret

→ metadata only

가 기본일 수 있다.

122. API에서 Raw Chain-of-Thought 금지

다음 Endpoint를 제공하지 않는다.

GET /executions/{id}/chain-of-thought

대신 Structured Decision Trace를 제공한다.

123. Execution Trace API

권한이 있는 경우 다음을 조회할 수 있다.

- action selected
 - tool call
 - model metadata
 - knowledge references
 - policy result
 - error reason
 - evaluation
-

124. Data Classification 전달

민감 Data를 처리하는 API에서는 Classification을 Resource Metadata 또는 Context로 전달할 수 있다.

Client가 Classification을 낮게 보내면 서버가 그대로 신뢰하지 않는다.

Server Policy가 검증한다.

125. API 입력 Validation

모든 입력은 Schema Validation을 거친다.

검증:

- Type
- Required
- Enum
- Length
- Format
- Reference existence

Application이 미리 Validation해도 Server Validation은 생략하지 않는다.

126. Unknown Field

엄격한 Contract에서는 알 수 없는 Field를 Error 처리할 수 있다.

또는 명시된 Forward Compatibility 규칙에 따라 무시할 수 있다.

Resource별 정책을 일관되게 적용한다.

127. Nullable와 Missing

다음은 구분될 수 있다.

```
{
```

과:

```
{
```

```
  "field": null
```

```
}
```

API Schema에서 의미를 명확히 정의한다.

128. Timestamp

Timestamp는 ISO 8601 형식을 사용한다.

예:

2026-08-21T03:28:00Z

API 내부에서는 UTC 사용을 권장한다.

UI가 Local Time으로 표시할 수 있다.

129. Enum

Enum 값은 Stable API Contract다.

Client가 임의 문자열을 사용하지 않는다.

Unknown Enum에 대한 Forward Compatibility 정책을 SDK에서 고려할 수 있다.

130. Numeric Precision

Cost, Usage 또는 비율 등 정밀도가 중요한 값은 Floating Point 표현 문제를 고려한다.

특히 Billing Amount는 Decimal 또는 Minor Unit을 사용할 수 있다.

131. Monetary Amount

상업 API 예:

```
{  
  "amount": "12.50",  
  "currency": "USD"  
}
```

또는:

```
{  
  "amount_minor": 1250,  
  "currency": "USD"  
}
```

정확한 Contract를 일관되게 사용한다.

132. Partial Update

PATCH에서 Field를 생략하는 것과 삭제하는 것을 구분해야 한다.

예:

- Missing → no change
- null → clear

정확한 의미는 Resource Schema에서 정의한다.

133. Bulk API

대규모 작업이 필요한 경우 Bulk Endpoint를 제공할 수 있다.

하지만 요청 하나에서 무제한 Resource를 처리하지 않는다.

대규모 Bulk는 비동기 Operation으로 처리하는 것이 좋다.

134. Bulk Error

일부 성공/일부 실패가 가능한 API는 결과를 명확히 반환해야 한다.

예:

```
{  
  "succeeded": [...],  
  "failed": [...]  
}
```

Transaction semantics를 문서화한다.

135. Import API

Resource Import는 단순 JSON Create보다 복잡하다.

확인:

- Schema Version
 - Dependencies
 - Tenant Mapping
 - Conflict
 - Secrets
-

136. Export API

Export에는 Secret 원문을 기본 포함하지 않는다.

Reference 또는 Secret Manifest만 제공할 수 있다.

137. API Compatibility

Minor Additive Change 예:

- Optional Field 추가
- 새 Endpoint

Breaking Change 예:

- Required Field 변경
- 기존 Field 의미 변경
- Enum 제거

Breaking Change는 API Version 또는 공식 Migration을 요구한다.

138. Deprecation Header

지원하는 경우 Response Header 또는 Documentation으로 Deprecated API를 알릴 수 있다.

Client는 Release Note만 의존하지 않고 Deprecation 신호를 모니터링할 수 있다.

139. SDK 선택 기준

SDK 사용을 권장하는 경우:

- 인증 처리
- Pagination
- Streaming
- Error Handling

- Type Safety
- Retry

직접 HTTP 사용은 완전히 허용되지만 API Contract를 그대로 따라야 한다.

140. SDK와 API Version

SDK Version과 API Version은 일치하는 숫자일 필요가 없다.

SDK는 지원하는 API Version 범위를 명시해야 한다.

141. CLI와 API

CLI는 API의 별도 우회 경로가 아니다.

기본 구조:

CLI

→ AGEX API

Admin CLI도 공식 Internal Operations Contract를 사용해야 한다.

142. API Usage Example — 최소 Agent 실행

1. Agent 생성

POST /api/v1/agents

2. Publish

POST /api/v1/agents/agt_123/publish

3. Execute

POST /api/v1/agents/agt_123/executions

4. 상태 조회

GET /api/v1/executions/exe_123

143. API Usage Example — Workflow + Approval

POST workflow execution

- execution queued
 - workflow running
 - approval requested
 - client gets approval
 - POST approve
 - workflow resumes
 - execution completed
-

144. API Usage Example — Knowledge 기반 Agent

Create Knowledge Collection

- Register Source
 - Start Ingestion
 - Wait Operation Complete
 - Bind Collection to Agent
 - Publish Agent
 - Execute Agent
-

145. API Usage Example — Plugin

Get Plugin

- Install Plugin
- Bind Credential
- Enable
- Attach compatible Tool to Agent

→ Execute

146. API Usage Example — Model Profile

Create Model Profile

- Configure allowed providers
 - Assign to Agent
 - Publish Agent
 - Execute
 - View selected model in Execution
-

147. API Usage Example — Webhook

Register Webhook

- Subscribe execution.completed
 - Start Execution
 - Receive Signed Webhook
 - Verify Signature
 - Process Event Idempotently
-

148. API Client 보안 체크리스트

Client는 최소 다음을 준수해야 한다.

- HTTPS
- Token 안전 저장
- Secret Log 금지
- Tenant Context 정확히 지정
- Error Detail 무분별 출력 금지

- Webhook Signature 검증
 - Retry 제한
 - Idempotency 사용
 - 최소 권한 Credential
-

149. Browser Client

Browser에서 AGEX API를 직접 호출할 수 있는 범위는 Security Architecture에 따라 제한한다.

특히 장기 Secret 또는 고권한 Service Credential을 Browser에 저장하지 않는다.

150. Backend-for-Frontend

고권한 작업이나 Secret이 필요한 경우:

Browser

→ Application Backend

→ AGEX API

방식을 권장할 수 있다.

151. CORS

CORS 허용 Origin은 명시적으로 관리한다.

*를 Production 고권한 API 기본값으로 사용하지 않는다.

152. API Gateway 보안

Gateway는 다음을 수행할 수 있다.

- TLS
- Authentication

- Rate Limit
- Request Size
- Request ID
- Basic Threat Filtering

하지만 Domain Authorization을 Gateway 하나에만 의존하지 않는다.

153. Server-Side Authorization

각 Domain Service는 자신의 Resource Permission을 재검증한다.

Gateway 통과가 Resource 접근 권한을 의미하지 않는다.

154. Runtime Enforcement

실제 Side Effect 직전 Runtime 또는 Plugin Gateway가 최종 Permission을 검증한다.

API 호출 시 한번 Permission을 확인했다고 실행 시점 검증을 생략하지 않는다.

155. Stale Approval 방지

Approval과 실제 실행 사이 Resource/Policy가 바뀌었으면 Runtime이 재검증한다.

필요 시 기존 Approval을 무효화한다.

156. API Rate-Limit Header

지원하는 경우 다음 정보를 제공할 수 있다.

- limit
- remaining
- reset

구체 Header 이름은 API Contract에 따른다.

157. Quota Error

예:

```
{
  "error": {
    "code": "QUOTA_EXCEEDED",
    "message": "Execution quota exceeded.",
    "details": {
      "quota_type": "concurrent_executions"
    }
  }
}
```

158. Policy Denial

Client는 Policy Denial을 일반 500 오류처럼 재시도하지 않는다.

사용자에게 적절한 사유를 표시하거나 관리자 승인을 유도한다.

159. Approval Required Response

일부 동기 API에서는 바로 실행하지 않고 Approval Resource를 생성할 수 있다.

예:

```
{
  "state": "approval_required",
  "approval_id": "apr_123"
}
```

일반적으로 Execution Resource와 연결하는 것이 더 일관적이다.

160. Async Operation Status

Operation 상태 예:

- queued
- running
- completed
- failed
- cancelled

Execution과 구분된 관리 작업이다.

161. Operation Result

대용량 Export나 Ingestion 완료 후 Artifact ID 또는 Result Reference를 반환할 수 있다.

162. Request Cancellation

HTTP 연결 종료는 서버 Operation Cancel과 동일하지 않다.

명시적인 Cancel Endpoint를 사용해야 한다.

163. Eventual Consistency

Search, Analytics 등 일부 API는 Resource Write 직후 결과가 약간 지연될 수 있다.

Transactional Resource 조회와 Search 조회의 일관성 기대를 구분한다.

164. Read-After-Write

Resource 생성 직후 GET /resource/{id}는 가능한 Strong Read를 제공하는 것이 일반적이다.

Search Index 반영은 지연될 수 있다.

165. API Audit

다음 API Action은 Audit 대상이다.

- Login/credential changes
 - Publish
 - Permission change
 - Policy change
 - Approval
 - Secret changes
 - Plugin install
 - Admin operation
-

166. Audit와 일반 Access Log

Access Log:

- HTTP 요청 중심.

Audit:

- 의미 있는 Security/Business Action 중심.

둘을 동일 데이터로 간주하지 않는다.

167. API Usage Tracking

Usage Meter는 API 요청 수뿐 아니라 Runtime 실제 사용량을 측정한다.

단순 API 200 횟수만으로 AI 비용을 계산하지 않는다.

168. Usage의 중복 처리

Event Retry나 API Retry 때문에 동일 논리 사용량이 이중 집계되지 않도록 Idempotent Metering을 적용한다.

169. API Error Monitoring

Application은 다음을 별도 모니터링하는 것이 좋다.

- 401/403 증가
- 409 증가
- 429
- 5xx
- Provider Error
- Execution Failure

170. Health API

일반적으로 다음과 같은 Endpoint를 제공할 수 있다.

/health

/ready

외부 Public Health와 내부 상세 Dependency Health를 구분할 수 있다.

171. API Documentation Source of Truth

공식 API 문서는 Machine-readable Contract를 기준으로 생성하는 것이 좋다.

예:

- OpenAPI 계열
- JSON Schema

수기 문서와 실제 API가 분리되지 않도록 한다.

172. Schema Code Generation

Canonical Schema를 기반으로 다음을 생성할 수 있다.

- SDK Types
- Validation
- API Documentation

생성물 자체를 다시 수동 Source of Truth로 만들지 않는다.

173. API Naming 원칙

권장:

- plural nouns
- lowercase
- 일관된 hierarchy

예:

/agents

/workflows

/executions

동일 의미의 Endpoint에 run, start, execute를 혼재시키지 않는다.

174. Action Endpoint

Resource Lifecycle이나 명확한 Command는 Action Subpath를 사용할 수 있다.

예:

/agents/{id}/publish

/executions/{id}/cancel

Generic RPC Endpoint 남용은 지양한다.

175. Resource Relationship

Parent/Child URI는 실제 Ownership 관계가 명확한 경우에 사용한다.

지나치게 깊은 URI:

/a/b/c/d/e

를 피한다.

176. Reference by ID

Resource 간 연결은 일반적으로 ID Reference로 한다.

예:

```
{  
  "model_profile_id": "mdlprof_123"  
}
```

객체 전체를 중복 Embedding하지 않는다.

177. Resource Snapshot

Execution에는 실제 사용된 Version Snapshot Reference가 저장될 수 있다.

Client가 실행 후 최신 Agent를 조회해 당시 상태를 추측하지 않아도 된다.

178. API Permission 변화

API에 새 기능이 추가되면 새 Permission이 필요한지 검토한다.

기존 Broad Permission에 조용히 포함시키지 않는 것이 좋다.

179. Breaking Permission Change

기존 Client가 갑자기 권한 확대/축소되는 변경은 일반 API 변경처럼 관리해야 한다.

Security 변경은 Release Note와 Migration을 요구할 수 있다.

180. API Security 테스트

필수:

- Auth bypass
- IDOR/Object Authorization
- Cross-Tenant
- Rate Limit
- Input Validation
- Replay
- Webhook SSRF
- Secret Leakage

181. API Contract Test

SDK와 Server는 동일 Contract를 기준으로 자동 테스트한다.

Client Mock도 공식 Schema를 사용한다.

182. Sandbox API Environment

Developer가 Production 데이터 없이 API를 시험할 수 있는 Sandbox/Development Environment를 제공할 수 있다.

Sandbox Credential을 Production에서 사용하지 않는다.

183. Environment별 Endpoint

예:

Development host

Staging host

Production host

Base API Contract는 동일하게 유지한다.

184. Environment Data 분리

Development API Key로 Production Tenant에 접근할 수 없도록 환경 Boundary를 분리한다.

185. Production API 변경

Production에 임시 Debug Endpoint를 추가하지 않는다.

필요하면 Internal Operations API로 명시적으로 설계한다.

186. Admin API

Admin API는 다음 특성이 있다.

- 강한 Authentication
- JIT 가능
- Audit Mandatory
- Narrow Scope

일반 Public Client가 사용할 수 있는 Endpoint와 분리할 수 있다.

187. Support API

Tenant 지원을 위한 API가 필요하다면 Impersonation보다 Support Session 구조를 사용한다.

188. Support Session Header/Context

지원 세션에서는:

- support session id
- tenant
- reason

- operator

를 Audit Context에 포함할 수 있다.

189. Multi-Tenant API 안전성

모든 Tenant-owned Resource API는 최소 다음 테스트를 통과해야 한다.

Tenant A ID + Tenant B Resource ID

→ Access Denied

ID를 안다고 접근 가능해서는 안 된다.

190. API Client 장애 복구

Client가 Process Restart되어도:

- execution_id
- operation_id
- idempotency key

를 안전하게 저장하면 진행 상태를 복구할 수 있다.

191. Client State 최소화

Server Runtime State를 Client Local Memory로 복제해 Source of Truth로 만들지 않는다.

Client는 AGEX API의 Resource/Execution 상태를 기준으로 한다.

192. Webhook 누락 대비

Webhook만을 유일한 상태 소스로 사용하지 않는다.

필요 시 Execution/Event API로 상태를 재동기화할 수 있어야 한다.

193. Event Ordering

분산 Event는 항상 전체 순서를 보장하지 않을 수 있다.

Consumer는 Event Version, Timestamp, Resource Revision 등을 활용해야 한다.

194. Event Schema 변경

기존 Consumer가 깨지지 않도록 Additive Change를 우선한다.

Breaking Change는 새 Event Version을 고려한다.

195. Webhook Secret Rotation

새 Secret과 기존 Secret의 Grace Period를 둘 수 있다.

Receiver는 Rotation Window 동안 둘 다 검증할 수 있다.

196. API 비용 최적화

Client는 불필요한 다음 패턴을 피한다.

- 1초마다 전체 목록 Polling
 - 반복 동일 Knowledge Retrieval
 - 불필요한 큰 Expand
 - 동일 Execution 중복 생성
-

197. API Performance

AGEX API 자체 Latency와 Model/Runtime Duration을 구분한다.

Agent 실행이 20초라고 API Gateway가 20초 느린 것은 아니다.

198. Synchronous API 적합 영역

- Resource CRUD
- Short Validation

- Metadata Query

199. Asynchronous API 적합 영역

- Agent Execution
- Workflow Execution
- Bulk Ingestion
- Export
- Index Build
- Long Processing

200. API Guide Acceptance Criteria

본 API Guide는 최소 다음을 충족해야 한다.

1. API Base URL과 Version 원칙을 설명한다.
2. Authentication과 Authorization을 구분한다.
3. Tenant Context 전달 방법을 설명한다.
4. Resource CRUD와 Lifecycle 사용법을 설명한다.
5. Version과 Revision을 구분한다.
6. Optimistic Concurrency를 설명한다.
7. Idempotency를 설명한다.
8. Cursor Pagination을 설명한다.
9. 표준 Error 처리 방식을 설명한다.
10. Retry 가능한 오류를 구분한다.
11. Agent Execution을 설명한다.
12. Workflow Execution과 Approval을 설명한다.
13. Knowledge/Memory API를 설명한다.

14. Plugin/Model API를 설명한다.
15. Marketplace Entitlement와 Permission을 구분한다.
16. Streaming과 Execution State를 구분한다.
17. Event/Webhook 사용법을 설명한다.
18. Admin/Support API의 보안 원칙을 설명한다.
19. Multi-Tenant API 보안 원칙을 포함한다.
20. SDK/CLI와 API 관계를 설명한다.
21. Secret 처리 원칙을 포함한다.
22. API Usage/Audit를 포함한다.
23. 비동기 Operation과 Execution을 구분한다.
24. Raw Chain-of-Thought API를 금지한다.
25. 실제 구현 Contract와 충돌하지 않는다.

201. API 사용 금지사항

다음 방식으로 AGEX API를 사용하지 않는다.

- Provider API를 직접 호출해 Model Gateway 우회
- Tenant Context를 Client-side Filter로 대체
- Published Resource를 직접 Patch
- 동일 Idempotency Key를 다른 업무에 재사용
- 모든 4xx/5xx를 무제한 Retry
- Webhook Signature 검증 생략
- Browser에 Service Credential 저장
- Secret을 일반 Resource Metadata에 저장
- Approval 받은 Payload 변경 후 기존 Approval 재사용
- Execution 상태를 Client Local State만으로 판단

- Streaming 연결 종료를 Execution 취소로 간주
 - Audit Event 수정 시도
 - Entitlement를 Runtime Permission으로 해석
 - Offset Pagination으로 대규모 데이터 전체 순회 전제
 - Long-running Task를 단일 HTTP 연결에 의존
 - Cross-Tenant Resource ID 접근을 정상 기능으로 사용
 - Internal Stack Trace를 사용자에게 그대로 표시
 - Raw Chain-of-Thought 조회 Endpoint 요구
 - Admin API를 일반 User Credential로 사용
 - Production DB를 API 대신 직접 접근
-

202. API Client 구현 체크리스트

Application 개발자는 최소 다음을 확인한다.

Authentication

- Token 관리
- Refresh/Rotation

Tenant

- 올바른 Tenant Context

Reliability

- Timeout
- Retry
- Idempotency

State

- Execution ID 저장
- Async Status 처리

Security

- Secret Log 금지
- Least Privilege

Observability

- Request ID
- Correlation ID
- Error Code

203. Agent Integration 체크리스트

1. 사용할 Agent ID/Version이 명확한가.
2. Tenant가 올바른가.
3. Input Schema를 만족하는가.
4. Execution ID를 저장하는가.
5. Timeout과 Cancel UX가 있는가.
6. Approval 상태를 처리하는가.
7. Streaming 중단 후 상태 복구 가능한가.
8. Error Code를 분류하는가.

204. Workflow Integration 체크리스트

1. Workflow Version을 명시하는가.
2. Input Schema를 검증하는가.
3. Waiting 상태를 실패로 처리하지 않는가.
4. Approval/Event Wait를 지원하는가.
5. Cancel을 명시적으로 호출하는가.
6. Step Error와 전체 Execution Error를 구분하는가.

205. Webhook Integration 체크리스트

1. HTTPS Endpoint인가.
2. Signature 검증하는가.
3. Timestamp 확인하는가.
4. Replay 방지하는가.
5. Event ID Dedup하는가.
6. 빠르게 2xx 응답하는가.
7. 내부 처리 실패를 자체 Queue로 넘길 수 있는가.
8. Event API로 재동기화 가능한가.

206. API Guide의 최종 정의

AGEX API Guide는 AGEX의 Platform Resource와 Runtime Capability를 외부 Application이 실제로 사용할 수 있도록 API Specification의 공식 계약을 호출 패턴, 오류 처리, 비동기 실행, Security 및 Client 구현 관점으로 구체화한 문서이다.

AGEX API를 사용하는 모든 Client는 먼저 자신의 Identity와 Tenant Context를 명확하게 전달하고, 서버가 수행하는 Permission 및 Policy Decision을 최종 권한 기준으로 받아들여야 한다.

Agent와 Workflow 실행은 장기적으로 지속될 수 있으므로 일반적인 HTTP Request/Response의 수명과 Execution의 수명을 분리해야 한다. Client는 Execution ID를 기반으로 상태를 조회하거나 Event 및 Streaming을 사용해야 하며, Network 연결 종료를 실행 종료로 간주해서는 안 된다.

Resource 수정에는 Version, Revision 및 Optimistic Concurrency를 사용하여 Silent Overwrite를 방지하고, 재시도에 의해 중복 Side Effect가 발생할 수 있는 작업에는 Idempotency를 적용해야 한다.

Model 호출은 Model Gateway를 통해 수행하고 Plugin Action은 Runtime Permission, Credential, Side Effect 및 Policy 검증을 거쳐야 하며, Application이 직접 Provider나 External Credential에 연결하여 AGEX의 통제 경로를 우회해서는 안 된다.

Knowledge, Memory, Marketplace, Governance, Audit 및 Usage API 역시 동일한 Tenant 와 Security 원칙을 적용하며, Commercial Entitlement은 기술적 실행 Permission과 분리 한다.

Streaming, Event 및 Webhook은 실시간 Integration을 제공하지만 최종 상태의 Source of Truth는 AGEX의 Resource 및 Execution API이다.

API Error는 안정적인 Error Code와 Request ID를 기반으로 처리하며 모든 Error를 자동 Retry하지 않는다. 특히 Permission과 Policy Denial은 시스템 장애가 아니므로 우회나 반복 호출의 대상이 되어서는 안 된다.

AGEX API Guide는 다음 문장으로 최종 정의한다.

AGEX API Guide는 외부 Application이 AGEX의 Resource와 AI 실행 능력을 Tenant·Identity·Version·Schema·Permission·Policy라는 명시적인 계약 아래 안전하고 재 시도 가능하며 추적 가능한 방식으로 사용하는 공식 Integration 기준이다.

본 문서는 AGEX API의 인증, Tenant Context, Resource Lifecycle, Execution, Streaming, Error, Idempotency, Event, Webhook 및 Client Security 사용 방식을 규정하는 공식 API Guide로 사용한다.